Lifting Large Language Models on Kubernetes

Four strategies to deploy models efficiently

Roland Huß Distinguished Engineer, Red Hat

Generative AI on Kubernetes

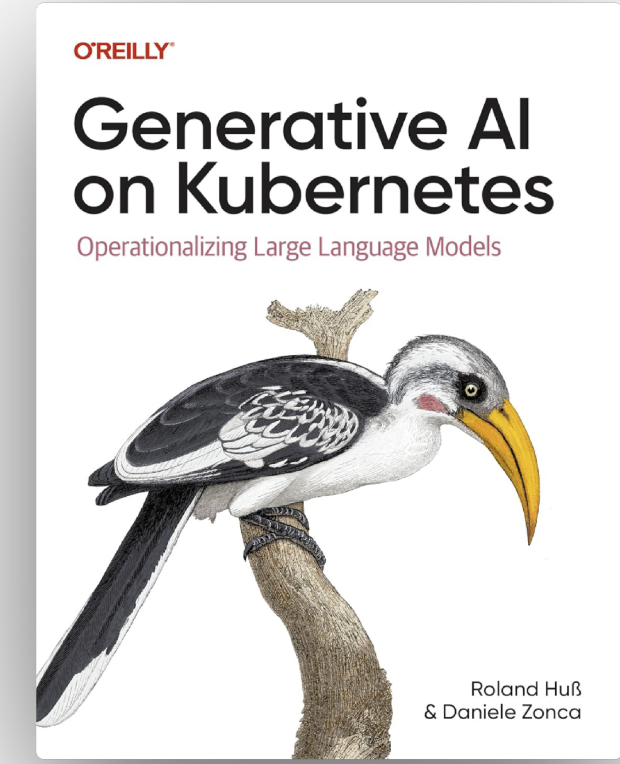
Roland Huß · Daniele Zonca

O'Reilly Media · March 2026

Chapter 3: Model Data

📖 **Early Release: Raw & Unedited**

bit.ly/genaik8s-raw



The Cold Start Problem

LLMs are 20-1500x larger than containers

Model	Parameters	Size (FP16)	vs Container
Typical container	-	**0.5 GB**	**1×**
Qwen 2.5 7B	7 billion	**15 GB**	30×
GPT-OSS 20B	21 billion (MoE)	**42 GB**	84×
Mixtral 8x7B	47 billion (MoE, 13 billion active)	**90 GB**	180×
Llama 3.1 70B	70 billion	**140 GB**	280×
DeepSeek V2	236 billion (MoE, 21 billion active)	**472 GB**	944×
Llama 3.1 405B	405 billion	**810 GB**	1620×

Three Strategies for Model Loading

🚀 Copy & Go

Init-container copies to shared volume

📦 Share & Scale

PersistentVolume shares across pods

🎯 Standardize & Distribute

OCI-based model distribution

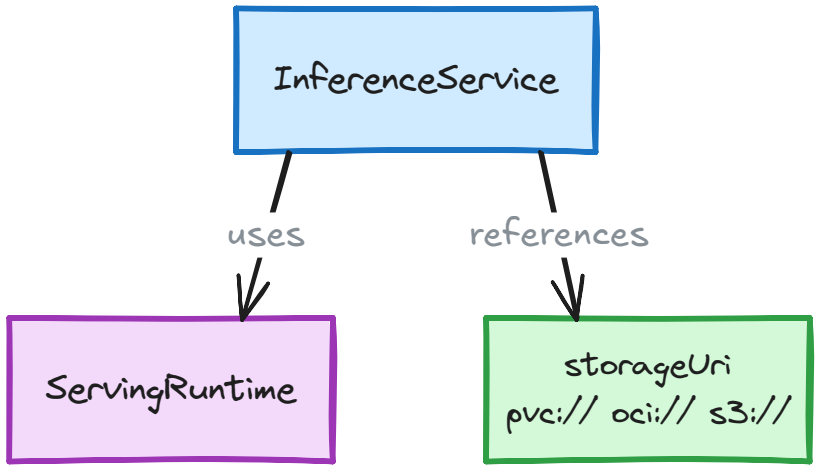
KServe simplifies model deployment on Kubernetes

What is KServe?

- Kubernetes-native platform for serving ML models
- Manages deployment lifecycle automatically
- Handles storage, health checks, metrics, routing

Key Concepts

- **ServingRuntime**: Template for model server configuration
- **InferenceService**: Declarative model deployment



Storage Abstraction

- Single interface for all model sources
- Flexible backends: **pvc://**, **oci://**, **s3://**, …

Declarative Model Deployment

ServingRuntime

```yaml
apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: pytorch-runtime
spec:
  containers:
    - name: kserve-container
      image: pytorch/torchserve:latest
      args:
        - torchserve
        - --model-store=/mnt/models
```

InferenceService

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: llama-3-8b
spec:
  predictor:
    model:
      modelFormat:
        name: pytorch
      runtime: pytorch-runtime
      storageUri: pvc://llama-3-8b-pvc/model
```

Runtime Reference

`name: pytorch-runtime` → `runtime: pytorch-runtime`

InferenceService references ServingRuntime by name

Storage Abstraction

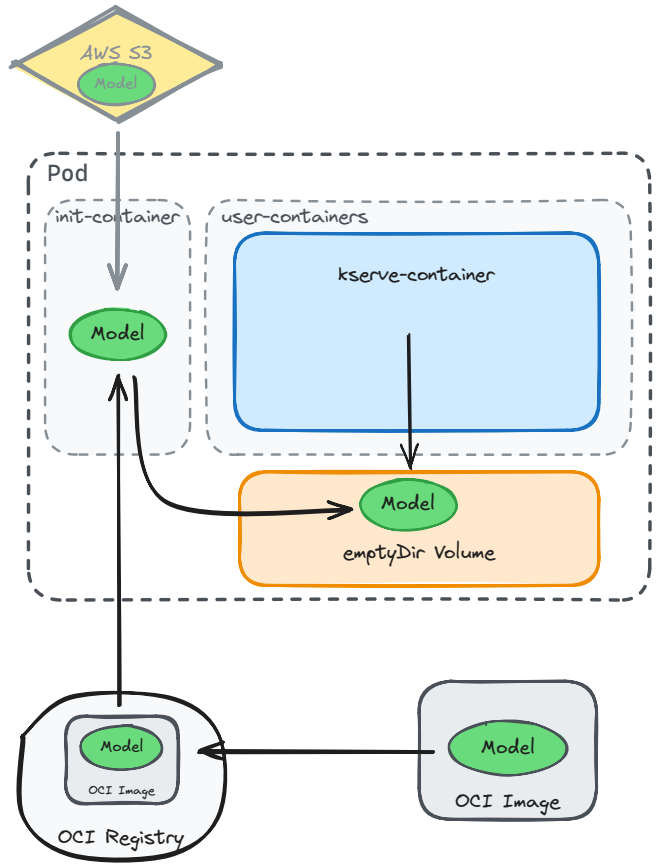
`storageUri: pvc://llama-3-8b-pvc/model`

Single field for all storage backends: `pvc://`, `oci://`, `s3://`, `gs://`, `hf://`

Init-container copies model before app starts

- **Simplest approach:** Copy model during pod startup
- **Flow:** Init runs → Copies → App reads
- **COPY step** is the bottleneck

```yaml
spec:
  initContainers:
  - name: model-loader
    image: registry.io/model:tag
    command:
    - cp -r /models/* /mnt/models/ # 5+ min!
  containers:
  - name: inference
    ...
```



Init-container trades simplicity for speed

```
spec:
  initContainers:
  - name: model-loader
    image: busybox
    command: ['sh', '-c']
    args:
    - wget -O /model/llama.bin https://models.example.com/llama-26gb.bin
    volumeMounts: [{name: model, mountPath: /model}]
```

🌟 Strengths

- Simple and well-understood
- Works on any K8s version
- No special features required

⛔ Limitations

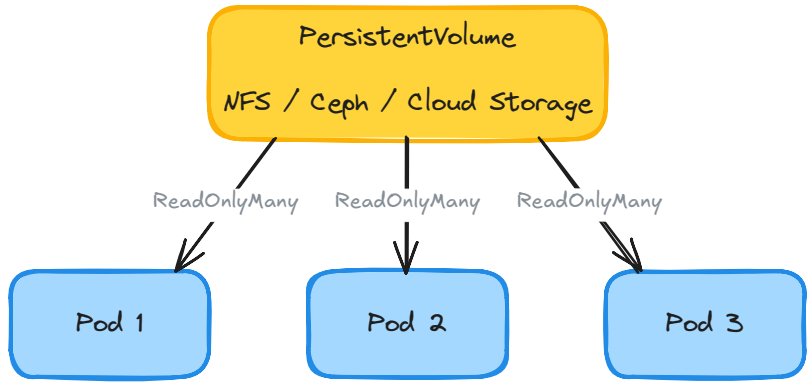
- Full copy every Pod start
- No sharing between Pods
- Slow startup (minutes)

Best For: Smaller models where simplicity > performance

Share Models with PersistentVolumes

Init-container **copies model for every Pod**

- 10 replicas = 10 copies
- Wastes node storage
- Repeated downloads



Single shared copy via PersistentVolume

- Network-backed storage (NFS, Ceph, cloud)
- All Pods mount same volume **read-only**
- Model uploaded once, consumed many times

Key Benefits

- **Storage efficient** for multiple replicas
- **Central updates** - change once, affects all
- **Separation of concerns** - data sci uploads, K8s consumes

PersistentVolumes share models across Pods

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: llama-3-8b-pvc
spec:
  accessModes:
    - ReadOnlyMany
  resources:
    requests:
      storage: 20Gi
```

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: llama-pvc
spec:
  predictor:
    model:
      modelFormat:
        name: pytorch
      storageUri: pvc://llama-3-8b-pvc/model
```

🌟 Strengths

- Storage efficient for multiple replicas
- Central model management
- Fast startup (just mount, no copy)

⛔ Limitations

- Network latency (distributed filesystem)
- Complex management
- Model discovery is hard

We already have infrastructure for distributing images

Question: Can we use OCI registries for model distribution too?

Discovery

Registries solve "where is the model?"

Versioning

Tag-based model versions

Distribution

Pull mechanisms + optimizations (prewarming, lazy loading)

New challenge: How do we access data from pulled images efficiently?

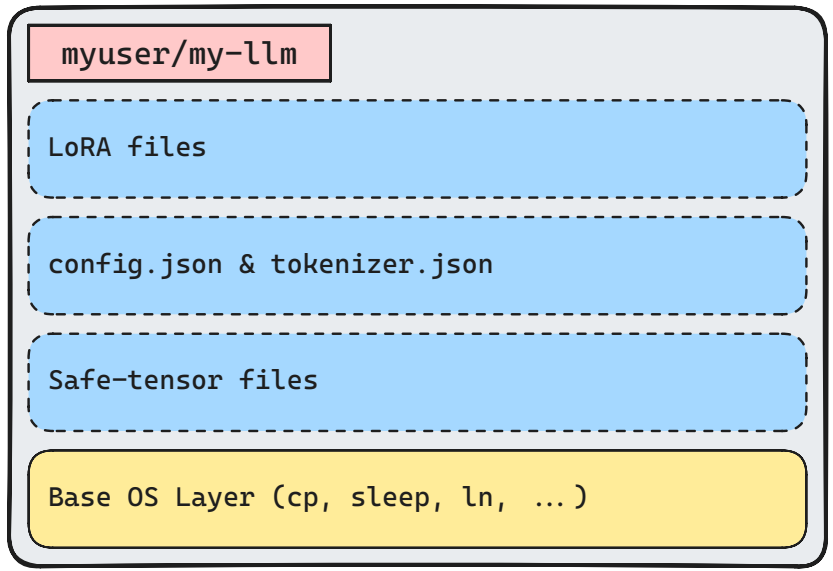
OCI's layered format enables zero-copy strategies

Registry solves distribution, not access

What OCI's layered format enables:

- **Layered storage** - shared base models
- **Layer reuse** - perfect for LoRA adapters
- **Caching** - download once per node
- **Foundation** for zero-copy access

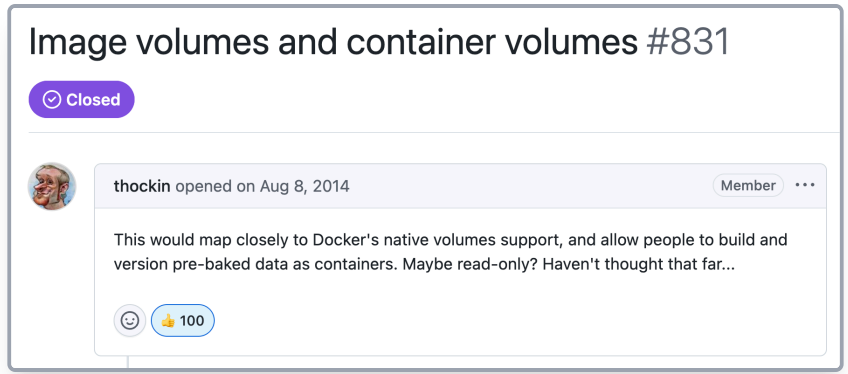
The challenge: Access pulled image data efficiently



Registries + layered format = foundation for zero-copy → Next: Strategies to access

Docker had image volumes from day 1 - Kubernetes waited 10 years

Year	Event
2013	Docker launches with `--volumes-from`
Aug 2014	K8s issue #831 opened
2014-2023	10 years of workarounds
Jul 2023	shareProcessNamespace hack
2024	K8s 1.31 ImageVolume



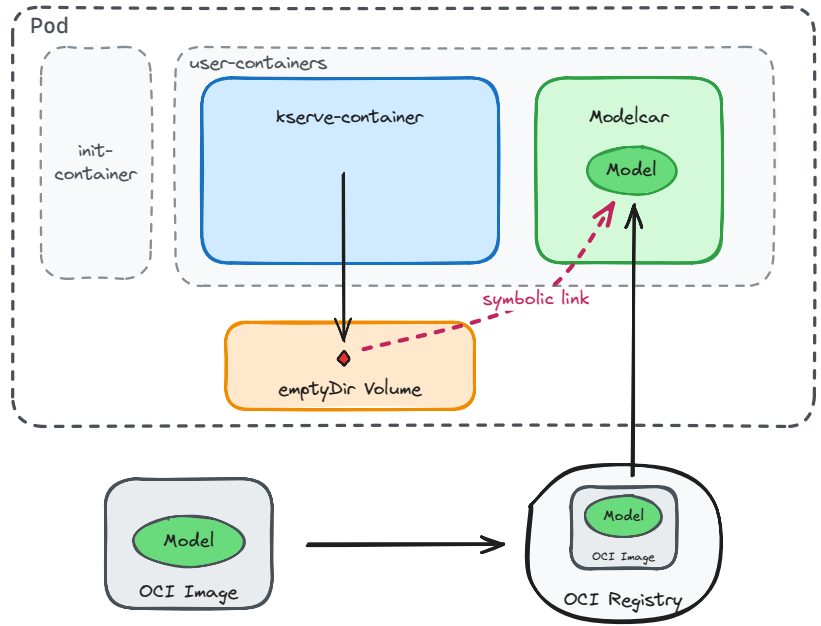
The Workaround

Discovery: `shareProcessNamespace` allows accessing another container's root filesystem

Applied to KServe → **modelcar** (2023)

Modelcar accesses image data via symlinks

- Modelcar runs as a **sidecar** container
- No init-containers needed
- Sidecar's container image **contains** the model
- Uses `shareProcessNamespace: true`
- Creates symlink to `/proc/<` *modelcar pid* `>/root/model`



KServe runtime follows symbolic link → No copy!

Modelcar eliminates copy overhead

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
spec:
  storageUri: oci://registry.example.com/models/llama-26gb:latest
  # Modelcar automatically creates symlinks from image layers
  # No init-container copying required
```

🌟 Strengths

- No copy - direct access via symlinks
- Fast startup (download once per node)
- Layer sharing across models
- Works on Kubernetes <1.31

⛔ Limitations

- Implementation complexity
- Security risks (shared namespaces)
- `shareProcessNamespace` must be enabled
- KServe-specific solution

Best For: KServe deployments on Kubernetes <1.31 needing fast startup

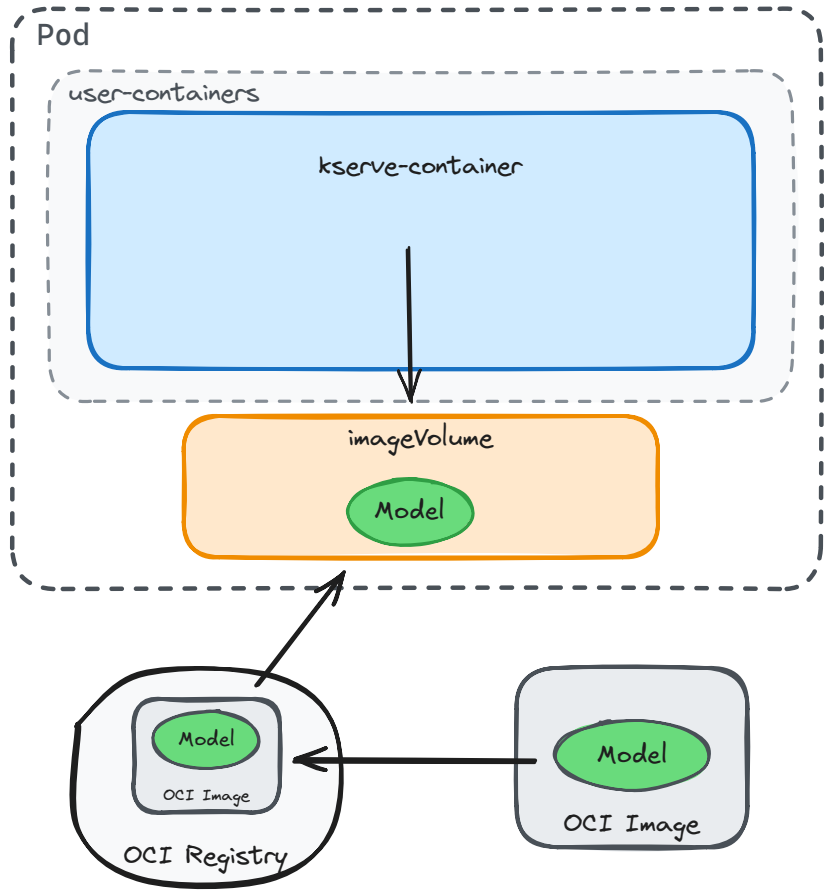
ImageVolume mounts OCI images natively

Kubernetes 1.31+ native feature:

- No init container
- No sidecar
- No copying
- **Direct mount** from OCI registry

Much simpler than modelcar!

Status: Beta in 1.33, enabled by default in 1.35 (Dec 2025)



ImageVolume extends Kubernetes' volume API

```yaml
spec:
  volumes:
  - name: model-data
    image:  # New volume type (1.31+)
      reference: registry.io/llama-7b:v1
      pullPolicy: IfNotPresent
  containers:
  - name: inference
    volumeMounts:
    - name: model-data
      mountPath: /mnt/models
      subPath: models  # Beta feature (1.33+)
```

- The OCI image **reference** is the only required field
- By default, the full root filesystem `/` gets mounted (read-only, no-exec)
- Use **subPath** to mount just a subdirectory like `/models`
- Standard image pull secrets work for authentication

ImageVolume is the native solution

🌟 Strengths

- Native Kubernetes API
- No copy overhead
- Simple YAML syntax

⛔ Limitations

- OCI images only (no artifacts)
- Runtime dependency:
  - containerd 2.1+
  - CRI-O 1.31+

📅 Feature Timeline

- 🟡 **Alpha** (1.31, Aug 2024): Feature gate required
- 🟡 **Beta** (1.33, Apr 2025): SubPath added, protected by feature gate
- 🟢 **Beta by default** (1.35, **Dec 2025**): No feature gate needed anymore
- ⏳ **GA**: Timeline TBD

Strategy choice balances speed and complexity

Strategy	**Speed**	**Complexity**	**K8s Version**	**Use When**
Init-container	❌ Slow	✅ Simple	Any	Small models
PersistentVolume	⚠️ Medium	⚠️ Medium	Any	Multiple replicas
Modelcar	✅ Fast	❌ Complex	<1.35	KServe + speed today
ImageVolume	✅ Fast	✅ Simple	1.35+	Future-proof

Three Things to Remember

1

LLM Deployment is a Data Problem

14-750GB breaks traditional K8s patterns

2

Four Paths with Clear Trade-offs

Simple vs fast, stable vs cutting-edge

3

Your Context Drives the Decision

K8s version, runtime, SLA matter more than "best practices"

The Future:

ImageVolume is the endgame